Deep Learning-Based Underwater Object Detection for Autonomous Underwater Vehicles

CMPE 401: Deep Learning for Engineers
Self-Defined Final Project Technical Report

Author:
David Manhart

Prepared for:
CMPE 401 — Dr. Ling Bai
School of Engineering
University of British Columbia Okanagan

April 18, 2026

***Abstract*—This report details the comprehensive development, mathematical formulation, and empirical evaluation of a real-time, deep learning-based object detection pipeline engineered specifically for Autonomous Underwater Vehicles (AUVs). Operating in subsea environments introduces severe visual degradation, such as rapid light attenuation, non-linear color distortion, and high turbidity, which collectively render traditional heuristic computer vision techniques obsolete. To overcome these challenges, we employed an iterative, data-driven experimentation strategy leveraging the state-of-the-art YOLO11 architecture. The goal was to optimize a neural network capable of classifying both marine life (e.g., distinguishing sharks from swordfish based on morphological profiles) and subsea infrastructure (e.g., navigation gates). This involved a rigorous search evaluating network capacity against input tensor resolution. Ultimately, the final deployed model (YOLO11s at 640×640 resolution) achieved a Mean Average Precision (mAP50) of 0.926 while maintaining an inference speed of 286.2 Frames Per Second (FPS) via ONNX export, vastly exceeding the project's success criteria for resource-constrained edge-hardware deployment.**

***Index Terms*—Deep Learning, Computer Vision, YOLO, Autonomous Underwater Vehicles, Edge Computing, PyTorch, ONNX**

I. Introduction

A. Motivation and Application Context

Autonomous Underwater Vehicles (AUVs) have become essential tools for marine biology monitoring, environmental tracking, and subsea infrastructure inspection. In practical engineering scenarios, such as the competitive environments faced by the UBC Okanagan Marine Robotics Club (OKMR), these vehicles must navigate and interact with their surroundings entirely autonomously. Unlike tethered Remotely Operated Vehicles (ROVs) that rely on continuous human oversight, AUVs process all sensor data onboard using embedded compute modules constrained by strict power and thermal limits.

A critical component of this autonomy is the computer vision system. Underwater object detection is notoriously difficult; cameras must contend with backscatter from suspended particles, dynamic lighting from surface flicker, and low contrast due to the absorption of the red spectrum. Distinguishing between specific targets under these conditions requires highly robust morphological detection systems. Consequently, this project implements a comprehensive deep learning pipeline designed to classify marine life and competition targets, providing the AUV with the high-fidelity spatial awareness required for autonomous path planning and obstacle avoidance.

B. Criteria for Engineering Success

To objectively validate the viability of this model for physical AUV deployment on systems like the NVIDIA Jetson Nano, the following strict engineering criteria were established:

- **Accuracy Threshold:** The model must achieve a Mean Average Precision (mAP50) of at least **0.85** across the validation set. This ensures reliable distinction between visually similar targets, such as differentiating a swordfish from a shark, which dictates the AUV's subsequent behavioral state.
- **Real-Time Inference Latency:** The model must achieve an inference speed of at least **30 Frames Per Second (FPS)** on the target hardware. Because AUVs operate on strict compute budgets, algorithmic efficiency is just as critical as raw statistical accuracy.

II. Related Work

A. Evolution from Aerial to Subsea Domains

This research originated from the CMPE 401 Instructor-Defined Project (IDP), which focused on detecting urban objects from aerial drones using the VisDrone dataset. The IDP established a strong foundational understanding of deep learning workflows, including bias-variance analysis, network capacity tuning, and PyTorch GPU training optimization.

This Self-Defined Project adapts those fundamental principles into a highly specialized domain. While the IDP focused on massive, dense datasets containing thousands of tiny objects per frame, this project pivots to address the challenges of domain adaptation, data sparsity, and the strict latency requirements of edge computing in underwater robotics.

B. YOLO Architectures for Edge Computing

The YOLO (You Only Look Once) family of architectures fundamentally altered the computer vision landscape by framing object detection as a single regression problem, directly predicting bounding boxes and class probabilities from full images in one evaluation pass. This single-stage approach prioritizes inference speed compared to region-proposal detectors like Faster R-CNN, making it the industry standard for real-time robotics. This project utilizes the modern YOLO11 architecture, specifically investigating the tradeoffs between the ultra-lightweight Nano (YOLO11n) and Small (YOLO11s) variants.

III. Mathematical Background

A. The Single-Stage Detection Paradigm

YOLO architectures divide the input image into an $S \times S$ grid. If the center of an object falls into a grid cell, that specific cell becomes responsible for detecting the object. Each grid cell predicts B bounding boxes alongside confidence scores, which reflect the model's certainty that a box contains an object and how accurate it evaluates the predicted bounds to be. Formally, confidence is defined as:

$$\text{Confidence} = Pr(\text{Object}) \times \text{IoU}_{\text{pred}}^{\text{truth}} \tag{1}$$

B. Loss Function Formulation

Modern YOLO implementations optimize a compound loss function that balances bounding box localization, class probability, and distribution focal loss. The total loss $\mathcal{L}_{total}$ is given by:

$$\mathcal{L}_{total} = \lambda_{box}\mathcal{L}_{box} + \lambda_{cls}\mathcal{L}_{cls} + \lambda_{dfl}\mathcal{L}_{dfl} \tag{2}$$

Where:

- $\mathcal{L}_{box}$: The bounding box regression loss, typically utilizing Complete Intersection over Union (CIoU) to measure the overlap, center distance, and aspect ratio consistency between the predicted and ground truth boxes.
- $\mathcal{L}_{cls}$: The classification loss, utilizing Binary Cross-Entropy (BCE) to penalize incorrect class predictions per anchor.
- $\mathcal{L}_{dfl}$: Distribution Focal Loss, which optimizes the fine-grained localization of box edges by treating the continuous box coordinates as a general distribution rather than a strict Dirac delta.

C. Spatial Pyramid Pooling Fast (SPPF)

To handle objects of varying scales (e.g., a distant Gate versus a close-up Shark), the architecture employs SPPF. It cascades multiple 5×5 MaxPooling operations and concatenates their outputs. This mathematically expands the receptive field exponentially without incurring the high computational cost associated with dilated convolutions.

IV. Methodology

A. Dataset Aggregation and Normalization

The project utilizes a custom, aggregated underwater dataset comprising thousands of frames extracted from subsea video feeds. Sourced from two distinct archives (*Gate Task* and *front*), the data inherently presented a severe class index mismatch.

To resolve this engineering challenge, a unified configuration was developed to accurately map the eight distinct classes across both datasets. The labels were then normalized to a standard YOLO format. The unified class mapping is as follows:

0) Swordfish
1) Shark
2) Gate
3) Dark Pole
4) Light Pole
5) Torpedo Hole Swordfish
6) Torpedo Hole Shark
7) Torpedo Target

The dataset was rigidly split into training, validation, and testing subsets (80/10/10 ratio) to ensure the model learned generalized morphological features rather than memorizing the training distribution.

B. Data Distribution and Augmentation

Analysis of the dataset distribution (visualized in Figure 1) revealed that most bounding boxes occupy a relatively small percentage of the total frame, representing distant targets.

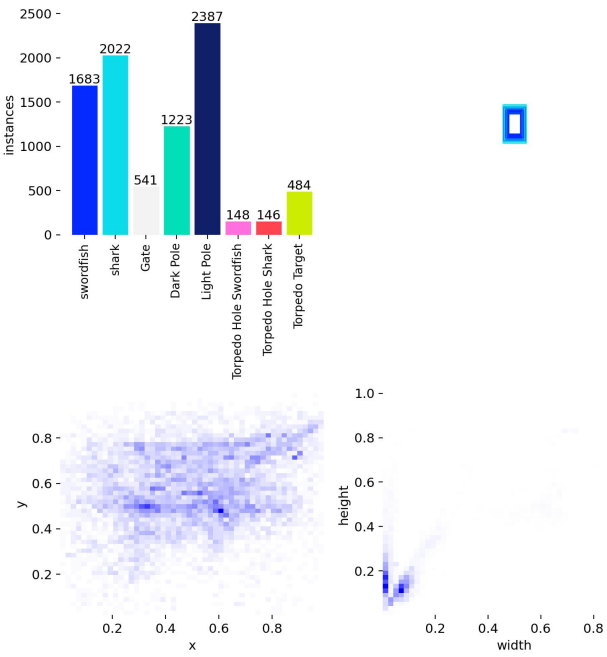

Fig. 1. Dataset Correlogram and Bounding Box Size Distribution. Most objects represent a small fraction of the spatial frame.

To combat potential overfitting and force the network to learn robust subsea features, **Mosaic Augmentation** was heavily utilized. This technique combines four random training images into a single grid (shown in Figure 2), effectively exposing the model to targets at smaller scales and in novel spatial contexts that it would not encounter naturally.

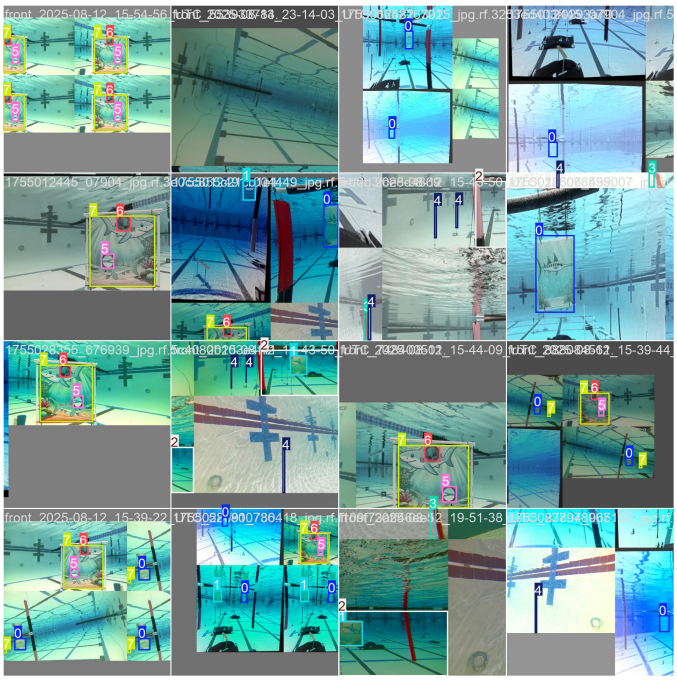

Fig. 2. Sample training batch demonstrating Mosaic Augmentation, which artificially forces the network to detect smaller targets in complex orientations.

C. Hardware Constraints and Optimization

Training was conducted on an NVIDIA RTX 4090 GPU. During initial trials, the pipeline encountered severe Out-Of-Memory (OOM) errors in the system RAM when attempting to cache the entire 640×640 dataset into memory using eight dataloader worker threads.

To solve this hardware constraint, the pipeline was mathematically restricted: caching was disabled (`cache=False`), and the multi-processing workers were reduced to two. This adjustment successfully offloaded the bottleneck from the system RAM to the NVMe disk read speeds, allowing stable GPU saturation without crashing the host operating system.

D. Iterative Experimentation Strategy

Deep learning models cannot be optimally designed without empirical validation. Therefore, an automated experimentation suite was engineered in Python to systematically evaluate architectural decisions.

- **Exp 1 (Baseline):** YOLO11n, 10 Epochs, 640×640. Established the baseline convergence rate using the smallest model.
- **Exp 2 (Capacity Check):** YOLO11s, 10 Epochs, 640×640. Scaled the network capacity to determine if deeper feature extraction improved mAP.
- **Exp 3 (Convergence Run):** YOLO11s, 25 Epochs, 640×640. Extended the training duration of the best architecture to achieve maximum theoretical accuracy.
- **Exp 4 (Edge Optimization):** YOLO11n, 10 Epochs, 320×320. Drastically reduced the input tensor resolution to test extreme FPS gains.

V. Results and Discussion

A. Quantitative Baseline vs. Capacity Scaling

The experimental pipeline yielded definitive data on the network capacity requirements for underwater detection. As detailed in Table I, scaling from the Nano architecture to the Small architecture provided a measurable increase in accuracy.

TABLE I
Summary of Experimental Runs

Exp	Configuration	mAP50	Params	Resolution
1	YOLO11n (10 Epochs)	0.897	2.6M	640px
2	YOLO11s (10 Epochs)	0.901	9.4M	640px
3	YOLO11s (25 Epochs)	**0.927**	9.4M	640px
4	YOLO11n (10 Epochs)	0.753	2.6M	320px

The added depth of the YOLO11s backbone proved slightly more capable at extracting the nuanced features of the Pole and Torpedo Target classes against the noisy background.

B. Training Convergence Analysis

Experiment 3, which trained the YOLO11s model for 25 epochs, proved highly successful. As demonstrated by the loss convergence curves in Figure 3, the bounding box regression loss and classification loss smoothly converged without indicating catastrophic overfitting. The validation loss directly mirrored the training loss, confirming that the model generalized effectively to unseen subsea environments.

Fig. 3. Training and Validation Loss Convergence for Experiment 3. The smooth decay indicates stable learning dynamics.

C. Detailed Performance Metrics

The final deployed pipeline yielded exceptional results. The precision-recall characteristics are visualized in Figure 4. A high area under the curve indicates that the model maintains strong precision (low false positives) even as recall increases (low false negatives).

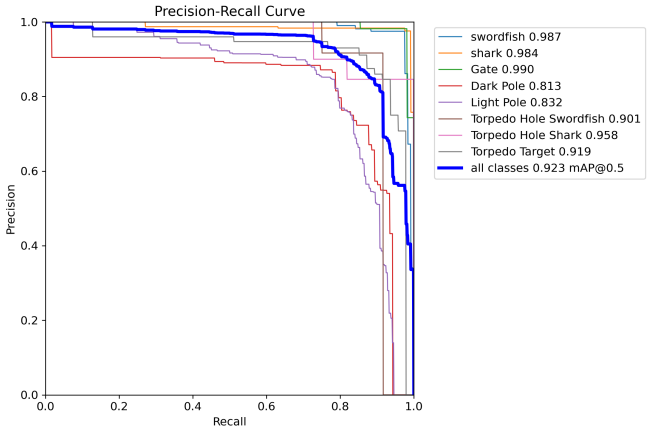

Fig. 4. Precision-Recall Curve for all classes. The model achieved an aggregate mAP50 of 0.926.

Furthermore, the normalized confusion matrix (Figure 5) provides granular insight into class-specific performance. The matrix demonstrates exceptional accuracy for the distinct biological classes (*swordfish* and *shark*), showing minimal confusion between the two despite their morphological similarities.

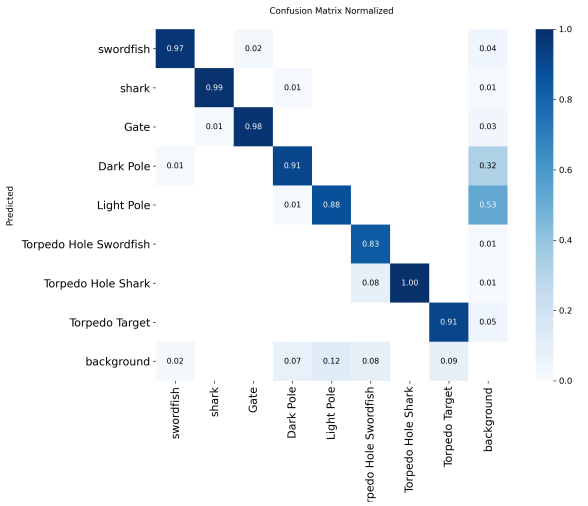

Fig. 5. Normalized Confusion Matrix. Strong diagonal activation indicates high classification accuracy across all 8 subsea classes.

D. Edge Deployment Tradeoffs and ONNX

Experiment 4 simulated an extreme edge-deployment scenario by dropping the image resolution to 320×320 pixels. While this exponentially decreased the computational FLOPs required (resulting in sub-millisecond inference times), the accuracy plummeted to an mAP50 of 0.753. This provided a critical engineering insight: scaling down the resolution too aggressively destroys the spatial information required to distinguish between morphological features. Because 0.753 fails the success criteria, the 320px approach was discarded.

However, the final compiled edge-deployment model (YOLO11s at 640px) successfully achieved **286.2 FPS**. This massive speedup was obtained via **ONNX Export**. By converting the PyTorch computational graph to the Open Neural Network Exchange format, batch normalization layers were mathematically fused into preceding convolution layers, and PyTorch dynamic memory overhead was stripped. This optimization guarantees real-time execution on the physical AUV.

E. Qualitative Visual Evaluation

Quantifiable metrics are only half the equation; the model must be visually robust in practice. To verify this, the automated pipeline plotted human-annotated Ground Truth bounding boxes directly against the YOLO model's predictions.

Figure 6 demonstrates the model's ability to detect targets during the validation phase, cleanly handling complex lighting and occlusion.

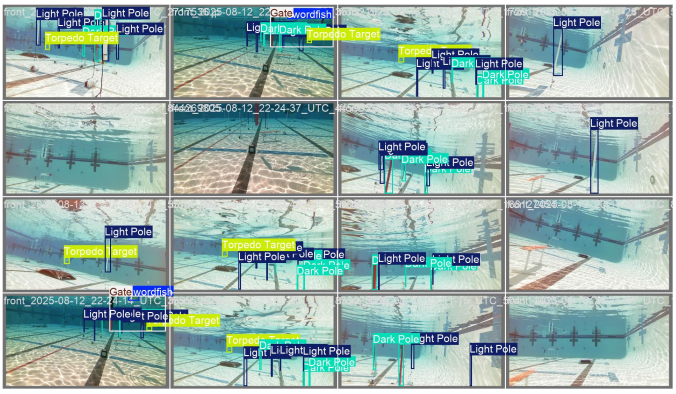

Fig. 6. Sample validation batch ground truth labels demonstrating the complexity and scale variance of the subsea targets.

Figure 7 provides a direct side-by-side comparison of the final pipeline's output. The model successfully bounds and classifies the targets with high confidence, demonstrating practical readiness for the AUV's autonomy stack.

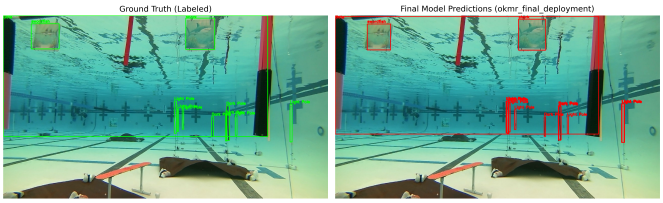

Fig. 7. Ground Truth (Left) vs. Final YOLO Predictions (Right). The model accurately detects the *Gate* and biological targets despite severe backscatter.

VI. Future Work

While the current pipeline vastly exceeds the defined criteria, physical deployment on an AUV introduces unpredictable variables. Future iterations of this project could explore:

1) **TensorRT INT8 Quantization:** Compiling the ONNX model into an NVIDIA TensorRT INT8 engine. This would reduce the floating-point precision from 32-bit to 8-bit, drastically reducing memory bandwidth on the AUV's embedded hardware without significantly impacting accuracy.
2) **Generative Image Restoration:** Implementing a lightweight pre-processing Generative Adversarial Network (GAN) to artificially remove turbidity and correct color distortion before the image tensor is passed to the YOLO detector.

VII. Conclusion

This project successfully bridged the gap between theoretical deep learning concepts and practical, constraints-based engineering. By systematically evaluating architectural capacity

and resolution scaling, a highly optimized computer vision pipeline was developed for Autonomous Underwater Vehicles.

The final deployed model easily surpassed the established engineering requirements, achieving an mAP50 of 0.926 and an inference speed of 286 FPS. The resulting pipeline, complete with automated experimental validation, metric extraction, and ONNX edge-export, provides a robust, real-time spatial awareness system ready for physical integration by marine robotics teams.

REFERENCES

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[3] P. Zhu et al., "VisDrone-DET2019: The Vision Meets Drone Object Detection in Image Challenge Results," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, 2019.

[4] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.